

A FIELD GUIDE

Guardrails for AI Agents

How a helpful agent learns to check itself, ask permission, and know its own limits, told through three small examples.

Input

Action

Risk

Based on the Guarded Agents in Action project

Contents

00	Before We Begin	PREFACE
01	The Guard at the Door	INPUT GUARDRAIL
02	Ask First, Act Later	ACTION GUARDRAIL
03	Not All Risk Is Equal	RISK ROUTING GUARDRAIL
04	The Pattern Underneath	HOW IT ALL FITS
05	The Road Ahead	WHAT COMES NEXT

Before We Begin

What is an agent, and what is a guardrail

An AI agent is a small program built around a language model. The model reads a request, thinks about it, and decides what should happen next. Sometimes it just replies with words. Sometimes it reaches for a tool and takes an action, such as looking something up or issuing a refund.

This is a powerful idea, but it raises a plain question. What happens when the model is given something it should not see, or asked to do something it should not do? A model cannot be expected to always notice this on its own. So we build a second layer around it. That second layer is called a guardrail.

Think of a road along the edge of a hill. The driver is skilled and the car is well built, yet the road still has a metal rail along the drop. The rail is not there because the driver is careless. It is there because a small mistake, made even once, should not lead to a disaster. A guardrail in an AI system plays the same part. It does not replace good judgment. It limits how far a bad moment can go.

Across the three examples in this book, one idea repeats again and again. A guardrail is not a clever trick and it is not part of the language model at all. It is an ordinary piece of code, often just a function with a few if statements, sitting quietly beside the model and checking its work.

THE IDEA IN ONE LINE

The model proposes. The guardrail decides. The application acts only after the guardrail agrees.

We will meet this idea three times, and each time it will grow a little. First the guardrail will watch what goes into the model. Then it will watch what the model wants to do. Then it will learn to tell the difference between a small risk and a large one. By the end, you will see that a safe agent is built the same way a safe building is built: layer by layer, with a plain inspection at every layer.

The Guard at the Door

An input guardrail keeps private data away from the model

Picture a customer writing to a support chat.

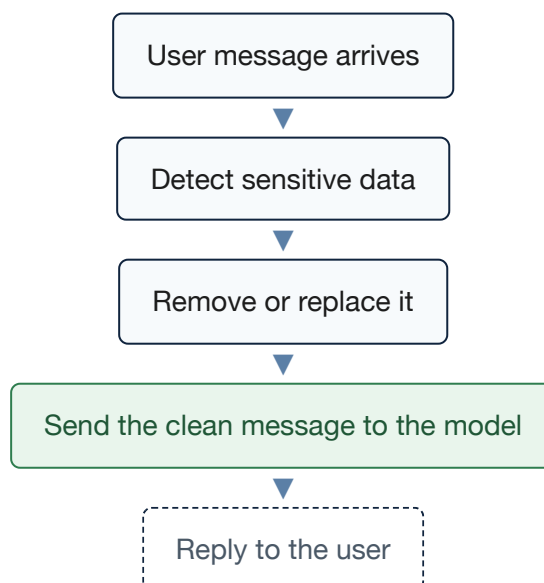
"My SSN is 123 45 6789. Why was my payment declined?"

The customer is not doing anything wrong. They are simply explaining their problem in their own words. But that sentence carries a social security number, a piece of information the language model never needs in order to help. Once such a detail is sent to a model, it has effectively left the building. The safest rule is also the simplest one: do not send it in the first place.

This is the job of an **input guardrail**. It stands at the door, reads every message before the model does, and removes anything sensitive on the way in.

A pipeline of three small steps

This first agent is built with LangGraph, a small toolkit for describing an agent as a set of steps, called nodes, connected by edges. A shared object, called the state, carries information from one node to the next. Nothing here is complicated. It behaves like a simple relay race, where each runner reads the baton, does one job, and passes it on.



Step one: detect

The first node asks a narrow question. Does this message contain a social security number, a card number, an email address, or a phone number. It does this with plain pattern matching, nothing more. The answer is stored as a short list, such as one entry that simply says `ssn`.

Step two: sanitize

The second node answers a different question. Given that a social security number was found, what should we do about it. In this project the policy is to replace it with a clear placeholder.

BEFORE	AFTER
My SSN is 123-45-6789.	My SSN is [REDACTED_SSN].

WHY SPLIT DETECTION FROM SANITIZING

Detecting a problem and deciding how to respond to it are two different jobs. Keeping them separate means the response can change later, perhaps blocking a card number outright while only masking a phone number, without touching the part of the code that finds them.

Step three: talk to the model

Only now does the message reach Claude. The model is told to act as a support agent, and it only ever sees the cleaned version of the text. The original message, social security number included, stays inside the application and is never part of the conversation sent to the model.

WHAT LANGGRAPH DOES, AND DOES NOT DO

LangGraph does not detect private data and it does not clean it either. It only moves the state from node to node in order. The real judgment lives in plain Python functions. LangGraph is the conveyor belt, not the inspector standing beside it.

In this first agent the belt always moves the same way. Every message passes through detection and sanitizing, even when nothing sensitive was found. There is no branching yet. That changes in the next chapter, where the path an action takes will depend on what the guardrail decides.

Ask First, Act Later

An action guardrail checks a decision before it becomes real

The first agent protected words going in. This agent protects something more serious: money going out.

"Please refund \$200 for invoice INV 1001."

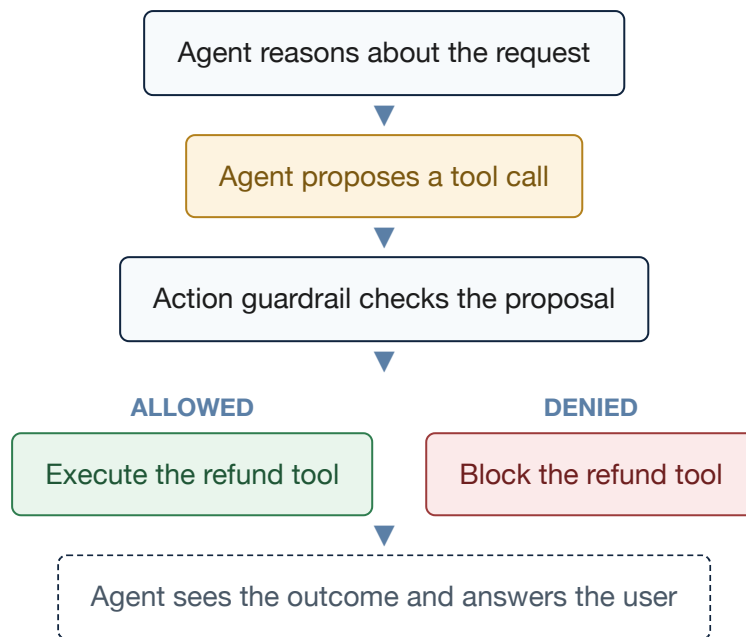
Here the model is given a tool named `issue_refund`. A tool is simply a function the model is allowed to ask for, along with arguments, such as an invoice number and an amount. This is often called a ReAct style agent, short for Reason and Act. The model reasons about the request, then proposes an action.

But notice the word propose. When the model asks for a tool to run, the refund has not happened. Nothing has moved yet. The model has only written down a plan. This one distinction is the whole chapter.

SELECTING A TOOL IS NOT THE SAME AS APPROVING IT

Claude may pick `issue_refund` as the right tool for the job. That is a language skill. Whether the refund should actually happen is a policy question, and policy questions belong to the guardrail, not the model.

Where the guardrail sits



The rule this guardrail applies today is deliberately small, so that the idea stays clear.

REQUESTED AMOUNT	DECISION
Zero or less	Blocked, not a real refund
Up to \$500	Allowed
More than \$500	Blocked, needs extra approval

A refund of \$200 passes this rule and the tool actually runs, producing a plain confirmation message. A refund of \$700 fails the rule. The refund function is never called. Instead, the model receives a short note explaining why, such as *Refunds above \$500 require additional approval*. Claude then relays this explanation to the customer, but it never gets to claim that a large refund went through, because it truly did not.

Why the path can now change

The first agent always moved in a straight line. This agent needs to make two decisions along the way: did the model ask for a tool at all, and if so, was that tool call allowed. LangGraph handles branching like this with what it calls conditional edges, which simply means the next step depends on the current state rather than being fixed in advance.

A SHORT LIST OF WHAT REAL GUARDRAILS ALSO CHECK

This example keeps things simple on purpose, using only the amount. A guardrail built for a real bank or business would likely also weigh customer identity, account status, daily limits, refund history, and signs of fraud, before ever letting an action run.

• • •

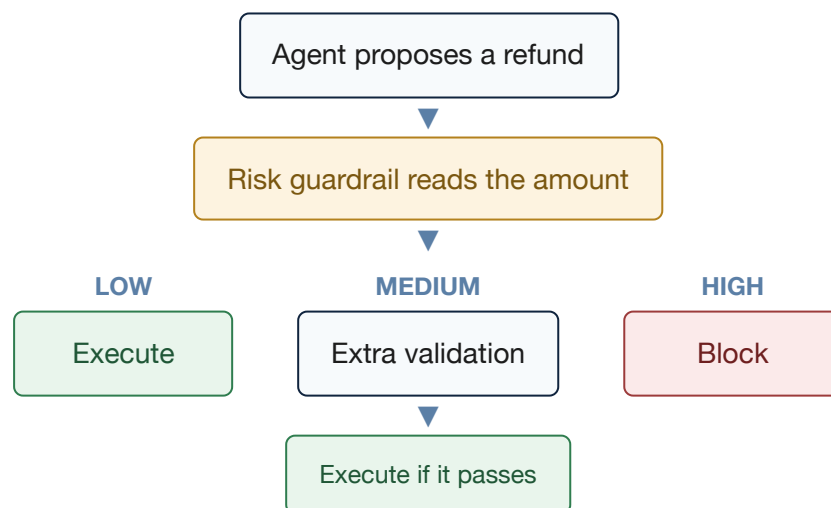
Not All Risk Is Equal

A risk based guardrail replaces a single gate with three doors

The previous chapter had only two outcomes, allow or block. That is a fair start, but real life rarely sorts so neatly. A refund of five dollars and a refund of five thousand dollars are not the same kind of event, even though both are, technically, refunds. This agent teaches the guardrail to tell them apart.

Instead of a single gate, the proposed action now walks through a classifier that sorts it into one of three levels.

AMOUNT	RISK LEVEL	WHAT HAPPENS
\$1 to \$100	Low	Runs right away
\$101 to \$500	Medium	Must pass one more check first
Above \$500	High	Blocked for now



The middle path deserves its own room

A medium risk refund is not simply nodded through. It is sent to one more node that performs an extra validation, in this case a plain check that the invoice number is present and correctly formatted, for example starting with the letters INV. Later this

same room could hold far stronger checks, such as confirming the invoice exists, was never refunded before, and matches the amount that was actually paid.

SAME TOOL, DIFFERENT RISK, DIFFERENT PATH

The tool being called is still just `issue_refund` every time. What changes is the road it travels down to get there. A cheap action can move quickly. An expensive one is made to slow down and prove itself.

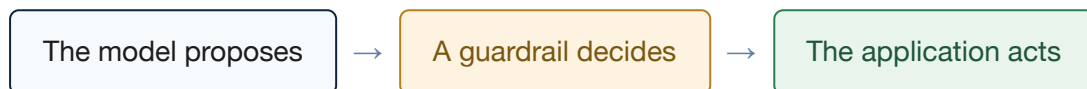
High risk requests are blocked in this version of the agent, the same way they were blocked in the previous chapter. But notice the wording used throughout this project: blocked for now. A block is not always the final word. Sometimes it simply means the decision needs a person, not a program. That thought carries directly into what comes after this book's examples, where a paused, high risk action can be handed to a human for approval instead of being refused outright.

• • •

The Pattern Underneath

Three agents, one repeated idea

Step back from the details of SSNs, refunds, and risk levels, and the same shape appears three times in a row.



Each chapter only changed where this pattern was placed and how detailed the decision became.

- **Chapter 01** placed the check at the very front door, on the words entering the model.
- **Chapter 02** placed the check right before a tool ran, on the action the model wanted to take.
- **Chapter 03** kept the check in the same place, but taught it to answer with a level of risk instead of a flat yes or no.

A few habits worth remembering

Deny by default

In every guardrail we have seen, an unknown or unexpected case is treated as unsafe rather than harmless. A tool that is not on the approved list is refused. An amount that makes no sense is refused. It is far cheaper to ask an agent to try again than to clean up after a mistaken action that already happened.

Never trust the model's own word about success

Every agent in this book is told, in plain language, not to claim a refund succeeded unless the tool itself confirms it. This matters because a language model can write a very confident sentence about something that never actually took place. The guardrail and the tool are the only parts of the system allowed to report the truth.

Guardrails are ordinary code

None of the checks in this book use anything mysterious. They are small functions, a handful of comparisons, and a couple of lookup rules. The intelligence is in the model. The safety is in the plain code sitting next to it. Keeping these two responsibilities in separate files, as this project does, makes each one easier to read, easier to test, and easier to trust.

ONE SENTENCE TO CARRY FORWARD

A capable agent and a safe agent are not the same achievement, and the second one is earned through layers of plain, boring, reliable checks.

• • •

The Road Ahead

Three guardrails down, several more to build

The three agents in this book are the opening pages of a longer project. The guardrail family they belong to keeps growing, and each future member protects a different part of an agent's life, from what it remembers to what it finally says.

- 01 · Input guardrail — covered in this book
- 02 · Action guardrail — covered in this book
- 03 · Risk based routing — covered in this book
- **04 · Human approval — pause a risky action and let a person decide**
- 05 · Memory guardrail — stop sensitive data from settling into long term memory
- 06 · Retrieval guardrail — check documents the agent reads for trust and access rights
- 07 · Plan guardrail — review an entire plan before any step of it is allowed to run
- 08 · Output guardrail — inspect the final reply before it ever reaches the user
- 09 · Blast radius guardrail — watch the total effect of many small actions added together

That final point is worth sitting with for a moment. One approved refund of two hundred dollars is a small, ordinary event. One hundred such refunds inside a single hour are not small at all, even though every one of them passed its own individual check. A guardrail that only looks at one action at a time can still miss a pattern building up across many of them. Watching the total, not only the parts, is the last piece of the picture.

CLOSING THOUGHT

A road needs more than one guardrail along its length, placed wherever the ground happens to fall away. An agent is no different. The goal was never to build a system that cannot make a mistake. The goal is a system where no single mistake, by the model or by chance, is ever allowed to travel very far on its own.

T h e E n d